DDRAM-O Milestone 1 Specification

Simplified Orthogonal-View Device and Timing Model

DDRAM Project

Revision 0.7 — 21 October 2026

Status. This document defines the first implementable DDRAM-O model for Ramulator2. Architectural interfaces and state transitions are normative. Electrical timing values introduced specifically for DDRAM-O are placeholders and must remain configurable until replaced by SPICE-calibrated values.

Contents

1 Purpose and Scope 1
- 1.1 Normative language . . . 1

2 Goals and Non-Goals 1
- 2.1 Milestone 1 goals . . . 1
- 2.2 Explicit non-goals . . . 1

3 Architectural Model 2
- 3.1 Views and addressing . . . 2
- 3.2 Data layout and operation orientation . . . 2
- 3.3 Per-bank state . . . 2
- 3.4 Simplified concurrency rule . . . 2

4 Command Interface 2
- 4.1 Required commands . . . 2
- 4.2 Multi-row activation abstraction . . . 2
- 4.3 Reserved primitive timings . . . 2
- 4.4 PuD command and operation reference . . . 2

5 Timing Model 3
- 5.1 Parameters and initial values . . . 3
- 5.2 Pumulator provenance and copy profiles . . . 3
- 5.3 Provisional timing ledger and rationale . . . 3
- 5.4 Applicability limits . . . 4
- 5.5 Activation-like command class . . . 4
- 5.6 Required timing relationships . . . 5
- 5.7 Activation power window . . . 5

6 Prerequisites and State Transitions 5
- 6.1 Conventional requests . . . 5
- 6.2 Atomic multi-row activation . . . 5

7 Layout-Aware Controller Integration 5
- 7.1 Request contract and ownership . . . 6
- 7.2 Queue organization and arbitration . . . 6
- 7.3 PuD queue lifecycle . . . 6
- 7.4 Command mapping and scheduling pipeline . . . 7
- 7.5 Commitment, dependencies, and completion . . . 7
- 7.6 Incremental integration and validation . . . 7

8 Configuration Interface 8

9 Trace and Statistics Requirements 8

10 Verification Plan 8

11 Calibration and Evolution 8
- 11.1 SPICE calibration . . . 8
- 11.2 Intersection-aware extension . . . 8
- 11.3 Functional PuD extension . . . 8
- 11.4 Intermediate-result cost accounting . . . 8

12 Implementation Boundary 9

1 Purpose and Scope

DDRAM-O provides two access structures over the same DRAM cell array. View A uses the conventional row orientation; view B is rotated by 90°, so a row in one view geometrically corresponds to a column in the other. The complete architecture creates cell-intersection hazards between simultaneously open rows. Milestone 1 intentionally omits those hazards and models the two row paths as independent, subject to an explicit cross-view activation delay.

The purpose of this milestone is to establish a stable device interface for:

- view-specific conventional accesses;
- overlap between activity on the two row paths;
- atomic processing-using-DRAM (PuD) primitives;
- configurable, visible placeholder timings; and
- command traces suitable for correctness and scheduling analysis.

This is a timing and state model. Functional arithmetic, operand-row allocation, energy calibration, and operation scheduling are later milestones.

1.1 Normative language

The words **must**, **must not**, **should**, and **may** describe implementation requirements. A requirement marked Provisional defines the initial model but may be replaced by calibrated data without changing the command interface.

2 Goals and Non-Goals

2.1 Milestone 1 goals

1. Introduce a distinct `DDRAM_O` DRAM standard.
2. Preserve the conventional DDR5 access path through view A.
3. Permit one conventional row to remain open independently in each view.
4. Introduce explicit activation, access, and precharge commands per view.
5. Introduce an atomic multi-row activation primitive per view.
6. Permit cross-view overlap after a configurable minimum delay.
7. Preserve conventional command-, column-, and data-bus sharing.
8. Make every DDRAM-O-specific timing overridable from Python configuration.

2.2 Explicit non-goals

Milestone 1 does *not* model:

- A/B row-intersection conflicts or simultaneous-sensing hazards;
- masked intersection bits or half-amplitude sensing;
- persistent states with three or five conventionally open rows;
- functional data values or arithmetic correctness;
- compute-row allocation, operand lifetime, or spilling;
- calibrated multi-row activation current or energy; or
- end-to-end CPU trace offloading.

Results produced by this model must not be described as SPICE-calibrated.

3 Architectural Model

3.1 Views and addressing

View A is the host-oriented conventional view. View B is the orthogonal view. Both commands use the existing hierarchical Ramulator address vector. In Milestone 1, the row field is interpreted in the coordinate system of the selected view. Consequently, row number r in A and row number r in B do not denote the same geometric row.

More precisely, the row field is a *view-local line index*. For example, `A:10` names line 10 in the A coordinate system, whereas `B:10` names line 10 in the orthogonal B coordinate system. The equal integers are labels in different namespaces; they do not mean that one physical row has been opened twice. The two selected lines may physically intersect at a cell, but Milestone 1 assigns that intersection no additional state transition, timing penalty, masking rule, or data-value effect. Thus both activations are legal in the simplified timing model. A later intersection-aware functional model must determine the shared-cell mapping and any resulting electrical or correctness restrictions.

The address mapper is not required to perform a physical transpose in this milestone. The selected command identifies the coordinate system. A later functional model will map both coordinate systems onto shared cells.

3.2 Data layout and operation orientation

The initial controller integration shall use view A for conventional host accesses and view B for bit-serial PuD. These are the baseline layout contracts, not interchangeable row-buffer placement choices. Bit-serial computation processes successive bit positions over time while operating on many independent values in parallel across sensing lanes. An operand's bit planes must therefore occupy the view-local lines expected by the PuD primitive, with matching lane identities for all operands.

For illustration, let $C[i, j]$ denote cells in a rectangular tile, where an A-line fixes i and a B-line fixes j. A compatible placement may store the bits of value x_i along A-line i:

$$C[i, b] = x_i[b], \qquad B_b = (x_0[b], x_1[b], \ldots).$$

Here B_b exposes a bit plane: lane i retains the same value identity as the bit index b advances. With several operands, separate B-lines hold their respective bit planes; a primitive combines corresponding lanes. This is an illustrative placement, not a fixed DDR5 address decoder or an assumption that one host value occupies an entire physical A-row.

Orthogonal access can expose an already compatible placement without an explicit transpose. It does not guarantee that arbitrary host buffers are compatible: packing, bit order, operand placement, tile geometry, and lane alignment must agree with the operation. Changing `selected_view` does not transpose, copy, or repack data. A layout mismatch requires an explicitly modeled conversion or rejection before execution; conversion costs must not be hidden in view selection.

Low-level `MRA_A` remains a supported device command. It does not make an arbitrary bit-serial PuD operation valid in A. Other operation/layout combinations may permit A only when their mapping and primitive semantics are explicitly defined. Likewise, `RD_B`/`WR_B` address B-local lines; they are not substitutes for host accesses to the same numeric A-row.

3.3 Per-bank state

Each physical bank maintains the state shown in Table 1.

The two active-row fields are independent. Opening or precharging one view must not change the other view's state. No equality check between the two row IDs is performed. A bank is conventionally closed only when both fields are closed.

Table 1: Required simplified state per physical bank.

Field	Domain	Meaning
`active_row[A]`	closed or row ID	Conventional row open through view A.
`active_row[B]`	closed or row ID	Conventional row open through view B.
`busy_until[A]`	cycle	Earliest cycle at which another A row-path operation may start.
`busy_until[B]`	cycle	Earliest cycle at which another B row-path operation may start.

3.4 Simplified concurrency rule

Commands on different views may overlap once their activation-like command starts are separated by `nXACTO`. Commands still contend for all shared resources inherited from DDR5. In particular, DDRAM-O adds row-path parallelism, not an additional off-chip data interface.

4 Command Interface

4.1 Required commands

Table 2 defines the Milestone 1 command set. "Local" means the state or timing effect applies only to the command's selected view.

4.2 Multi-row activation abstraction

`MRA_A` and `MRA_B` represent the complete sequence

$$\text{multi-row activation} \rightarrow \text{sense/majority} \rightarrow \text{restore} \rightarrow \text{precharge}. \tag{1}$$

The command must begin with its local view closed. If that view is open, its prerequisite is the corresponding local precharge. The opposite view may remain open. At command issue, the local view enters a timed busy state but does not acquire a persistent active-row identity. It is logically closed after the atomic operation.

Milestone 1 may use the command address as a representative bank/subarray target. The operand-row list belongs to the compound PuD request introduced in Milestone 2 and must not be encoded through unrelated address-vector fields.

4.3 Reserved primitive timings

The implementation should also define `nCOPY` and `nNOT` timing parameters now, even if explicit `COPY_A/B` and `NOT_A/B` commands are introduced with functional primitives in a later milestone. Reserving separate parameters prevents future calibration from coupling logically different primitives merely because their initial values are equal.

4.4 PuD command and operation reference

Table 3 is the reference for planned PuD commands and their layout-to-view mapping. Device primitives and compiler-level operations are distinguished explicitly: an operation name does not imply a new DRAM opcode. Layout compatibility follows Section 3.2; timing symbols refer to Table 4 and the profiles in Section 5.2. The timing ledger remains M1-P1.

Initial implementation target: PuD executes through view B; ordinary host accesses use view A. The highlighted B primitives define the initial PuD controller scope. Greyed A-view PuD primitives are deferred, optional extensions and must not be generated by the initial PuD compiler or selected by the controller. Existing low-level `MRA_A` device support and device tests remain valid. This deferral does not restrict ordinary A-view

Table 2: Required DDRAM-O commands.

Command	Scope	Semantics
`ACT_A`	Bank, local	Open the target row in view A.
`ACT_B`	Bank, local	Open the target row in view B.
`PRE_A`	Bank, local	Close only the row open in view A.
`PRE_B`	Bank, local	Close only the row open in view B.
`RD_A`, `WR_A`	Bank, shared data path	Read or write the row open in view A.
`RD_B`, `WR_B`	Bank, shared data path	Read or write the row open in view B.
`MRA_A`	Bank/subarray, local	Execute an atomic multi-row activation on view A; no persistent multi-row-open state remains.
`MRA_B`	Bank/subarray, local	Symmetric atomic multi-row activation on view B.
`PREab`	Rank	Close both views in all targeted banks, for maintenance compatibility.
`PREsb`	Same-bank scope	Close both views in the targeted banks, for maintenance compatibility.

reads, writes, or their prerequisites.

The initial model uses the same symbolic primitive cost in A and B; this is a provisional symmetry assumption, not evidence of equal circuit latency. At DDR5_3200AN under the default split profile, the reference values are `nMRA`= 76, `nCOPY`= 83, and `nNOT`= 83 cycles. The MAJ3 bit-plane baseline is consequently 408 cycles; it is not a complete multi-bit addition or multiplication. Explicit overrides and alternate copy profiles supersede these example numbers.

Service time is measured from primitive issue and includes the internal final precharge assumed by AP/AAP. A prerequisite precharge of an initially open view is additional. Shared bus/power limits, cross-view `nXACTO`, dependencies, and contention can delay issue or completion further. MRA's command/address occupancy remains two cycles, distinct from `nMRA`; COPY/NOT command encoding, occupancy, and prerequisite rules still require definition. They must join applicable activation constraints when implemented. Ordinary host and B-local read/write commands retain the DDR5 timing rules in Tables 2 and 6.

For a serialized graph with no additional movement, the primitive service sum is

$$L_{\text{seq}} = N_M\texttt{nMRA} + N_C\texttt{nCOPY} + N_N\texttt{nNOT},$$

where the counts refer to emitted leaves after placement and spill insertion. Count each inversion either as one NOT leaf or as its expanded copy sequence, never both. This sum is a service-cost reference; actual scheduled latency depends on graph dependencies and resource availability.

When a command or layout requirement changes, update this catalogue, its mapping/validation rules, and the corresponding acceptance tests together. Keep formulas in the timing ledger as the source of defaults; regenerate numeric examples for the selected preset after any timing/profile change. Increment the timing-ledger revision when timing assumptions change, and the document revision for catalogue or interface-only changes. Every future primitive must specify layout/operands, legal view, prerequisites, service and CA occupancy, completion semantics, and implementation status before it is enabled by the controller.

5 Timing Model

5.1 Parameters and initial values

Command delays are positive integer DRAM cycles; the decoder-delay input t_{DEC} is specified in nanoseconds. Table 4 distinguishes Pumulator-inherited primitive assumptions from DDRAM-O-specific placeholders. The current provisional timing ledger is revision M1-P1. This identifier and the resolved values should accompany experiment results until calibrated parameters replace the ledger.

The default `nXACTO` equals the current provisional DDRAM-P cross-view delay for comparability; it is not an electrical claim. The `nMRA` and `nCOPY` defaults inherit Pumulator's SIMDRAM AP/AAP cost model, with unit base and primitive scaling factors and ceiling conversion to cycles. This inheritance supplies an interim modeling baseline, not electrical validation for DDRAM-O. In particular, the copy default assumes a split row decoder and `nNOT` assumes a dual-contact-cell (DCC) inversion path; these circuit capabilities must be confirmed for the eventual device. SPICE calibration may replace the values without renaming commands.

5.2 Pumulator provenance and copy profiles

The reference is the supplied `pumulator_infra_all` source snapshot at commit `8fd8bd6f`. The inherited equations are implemented by `set_pum_latencies()` in `simulator/ramulator2/src/dram/impl/DDR5-PUM.cpp`; the matching defaults are in `simulator/ramulator2/configs/pum/ddr5/simdram.yaml`. With $t_{\text{RAS}} = \texttt{nRAS}\,t_{\text{CK}}$ and $t_{\text{RP}} = \texttt{nRP}\,t_{\text{CK}}$, they give:

$$n_{\text{AP}} = \left\lceil \frac{t_{\text{RAS}} + t_{\text{RP}}}{t_{\text{CK}}} \right\rceil,$$

$$n_{\text{AAP,split}} = \left\lceil \frac{t_{\text{RAS}} + t_{\text{DEC}} + t_{\text{RP}}}{t_{\text{CK}}} \right\rceil,$$

$$n_{\text{AAP,two-activate}} = \left\lceil \frac{2t_{\text{RAS}} + t_{\text{RP}}}{t_{\text{CK}}} \right\rceil.$$

The default maps `nMRA` to AP and `nCOPY` to split-decoder AAP. The two-activate profile (Pumulator's `AAPN`, configuration `aap_variant: naive`) is an alternative sensitivity assumption when a split decoder is unavailable; it sets `nCOPY`= 2`nRAS` + `nRP` and derives an unoverridden `nNOT` from that copy cost. Neither profile establishes a measured upper or lower bound on DDRAM-O latency.

Revision M1-P0 used the legacy Pumulator estimate `nRAS` + `nRP` + $\lceil$`nRP`$/4\rceil$, also found in `polypum/polypum.py`. Revision M1-P1 replaces its precharge-proportional margin with the graph-based model's explicit 4 ns decoder delay. This avoids making decoder delay implicitly track the precharge duration. Provenance here refers to the supplied implementation, not an independently verified circuit measurement or paper result.

5.3 Provisional timing ledger and rationale

Table 5 records why each placeholder exists and what evidence is required to retire it. The defaults are modeling hypotheses,

Table 3: PuD command/operation catalogue. Blue B-primitive rows are the initial PuD target; grey A-primitive rows are deferred optional extensions. Timings are provisional, not calibrated.

Command / operation	Layout and operand requirement	View	Service time	Status / remaining requirements
`MRA_B`	Bit-serial bit planes on B-local operand lines; matching value lanes.	B	`nMRA`	Timing-only device and controller path implemented, including service completion. Operand lists, physical layout validation, and functional majority pending.
`COPY_B`	Source and destination are B-local bit planes with identical lane and bit interpretation in the same local compute domain.	B	`nCOPY`	Initial PuD target; timing parameter reserved. Requires operand encoding, destination placement, and copy-path support. No transpose implied.
`NOT_B`	B-local source and destination with matching lanes; a DCC-compatible inversion destination/path.	B	`nNOT`	Initial PuD target; timing parameter reserved. Copy-equivalent default assumes DCC support; functional path pending.
`MRA_A`	Explicit A-local operand-line mapping for a supported operation; not the baseline bit-serial layout.	A	`nMRA`	Deferred for PuD integration. Low-level device timing/state exists; functional A-view PuD is an optional extension.
`COPY_A`	A-local source/destination with matching packed-bit interpretation in the same local compute domain.	A	`nCOPY`	Deferred optional extension; no opcode implemented. Requires a defined A-local copy path and operands.
`NOT_A`	A-local source/destination with matching bit interpretation and a supported DCC inversion path.	A	`nNOT`	Deferred optional extension; no opcode implemented. Requires independently confirmed A-local inversion support.
MAJ3 bit-plane operation	Three aligned B-local operand bit planes, temporary compute lines, and one destination bit plane.	B	3`nCOPY` + `nMRA` + `nCOPY`	Planned compiler sequence, not an opcode. Baseline preserves three inputs and copies one result; elision requires proven placement/liveness.
Addition / multiplication	Compatible bit-serial operands and intermediates, declared width/bit order, aligned lanes, and allocated temporary lines.	B	Dependency-graph cost; no fixed scalar timing	Planned compound operations. Decompose into MRA/COPY/NOT leaves; include carry/intermediate dependencies and required spills.
Layout conversion / cross-view movement	Explicit source and destination layout descriptors, coordinate mapping, and dependency ordering.	A/B	Unspecified; no timing parameter yet	Future explicit mechanism, not an implemented opcode. May be unnecessary for an already compatible placement; otherwise conversion is not free.

Table 4: DDRAM-O-specific timing parameters.

Parameter	Initial value	Status	Interpretation
`nXACTO`	8	Provisional	Minimum start-to-start delay between activation-like commands on opposite views of one bank/subarray.
`nMRA`	`nRAS` + `nRP`	Provisional	Local-view occupancy of an atomic multi-row activation, including restoration and precharge.
`nCOPY`	`nRAS` + `nRP` + n_{DEC}	Provisional	Pumulator split-decoder activate–activate–precharge (AAP) copy cost; $n_{\mathrm{DEC}} = \lceil t_{\mathrm{DEC}}/t_{\mathrm{CK}} \rceil$, initially $t_{\mathrm{DEC}} = 4$ ns.
`nNOT`	`nCOPY`	Provisional	Initial inversion cost when represented as a DCC-assisted copy.

not predictions of a fabricated DDRAM-O device.

The implementation must resolve the DDR5 preset and all base-timing overrides before deriving these values. Explicit primitive overrides take precedence and must be positive integer cycles. An unoverridden `nNOT` follows the final resolved `nCOPY`; overriding `nMRA` alone must not change `nCOPY`. For the current `DDR5_3200AN` preset, `nRAS`= 52 and `nRP`= 24 with $t_{\mathrm{CK}} = 0.625$ ns, yielding $n_{\mathrm{DEC}} = 7$, `nMRA`= 76 cycles (47.5 ns), and `nCOPY`=`nNOT`= 83 cycles (51.875 ns) under the split profile; the two-activate copy profile instead gives 128 cycles (80 ns). `nXACTO` remains 8 cycles. The old M1-P0 copy value was 82 cycles. The reserved `nCOPY` and `nNOT` values do not yet gate an issued command and therefore must not be interpreted as measured operation latency in Milestone 1 results.

5.4 Applicability limits

Pumulator does not establish DDRAM-O's orthogonal cross-view coupling delay, intersection behavior, MRA command encoding, or activation-current weight. Thus `nXACTO`, the two-cycle MRA CA occupancy, and the one-event MRA weight in `nRRD`/`nFAW` remain DDRAM-specific provisional assumptions, not Pumulator-inherited timings. A one-event weight does not limit MRA to four simultaneously activated rows; it approximates its demand on the shared activation-power budget.

Other Pumulator substrates must not be mixed into the default SIMDRAM profile. PRADA's 5 ns offset cancellation and 35 ns inversion, ELP2IM's pseudo-precharge and shortened restoration, and TRDRAM's shift costs require their respective circuitry. Short-lived intermediate values alone do not justify reducing the full restore interval of the current atomic MRA.

5.5 Activation-like command class

For cross-view constraints, the activation-like class is

$$\mathcal{A}_A = \{\texttt{ACT_A}, \texttt{MRA_A}\},$$
$$\mathcal{A}_B = \{\texttt{ACT_B}, \texttt{MRA_B}\}.$$

Future copy and inversion commands must join the class if their physical implementation begins with activation of their selected view.

Table 5: Provisional timing ledger, revision M1-P1.

Item	Current rule	Reason for the choice	Replacement evidence
`nXACTO`	8 cycles	Matches the DDRAM-P cross-view placeholder, giving initial A/B comparisons the same assumed coupling delay. It is deliberately independent of the DDR5 speed bin.	Worst-case SPICE separation between opposite-view sensing starts, converted with $\lceil t/t_{CK} \rceil$.
`nMRA`	`nRAS` + `nRP`	Inherited Pumulator AP cost: one complete active/restore interval plus local precharge. No extra precharge is charged after this atomic primitive.	Simulated multi-row sensing, restoration, and precharge completion time for each supported row multiplicity.
`nCOPY`	`nRAS` + `nRP` + n_{DEC}	Inherited Pumulator split-decoder AAP cost, with a separately configurable 4 ns decoder delay. Derived from DDR5 base timings, independently of any MRA-specific override.	Confirmation of split-decoder support and an end-to-end source-to-destination copy waveform, including safe local-view reuse.
`nNOT`	`nCOPY`	Inherited Pumulator SIMDRAM assumption: one AAP into a DCC row per inverter. Equality is conditional on that circuit path, not a general inversion latency.	Dedicated inversion-path latency, or evidence that the copy and inversion paths are identical.
MRA CA occupancy	2 cycles	Conservatively matches DDR5 address-bearing activation commands while the final multi-row operand encoding is unspecified.	Final command encoding and command/address bus protocol.
MRA activation weight	1 event	Lets existing `nRRD`/`nFAW` machinery limit MRA immediately, without claiming that MRA draws ordinary-ACT current.	Calibrated current envelope and either a weighted activation budget or dedicated MRA power constraint.

Table 6: Normative timing relationships.

Preceding	Following	Delay	Scope
$\mathcal{A}_A$	$\mathcal{A}_B$	`nXACTO`	Same bank/subarray, cross-view
$\mathcal{A}_B$	$\mathcal{A}_A$	`nXACTO`	Same bank/subarray, cross-view
`MRA_A`	Any `A` row command	`nMRA`	Local view
`MRA_B`	Any `B` row command	`nMRA`	Local view
`ACT_A/B`	Local `RD/WR`	`nRCD`	Inherited DDR5 behavior
`ACT_A/B`	Local `PRE`	`nRAS`	Inherited DDR5 behavior
`PRE_A/B`	Local activation-like	`nRP`	Local view
`RD_A/B`, `WR_A/B`	Column commands	DDR5 column constraints	Shared bank/channel data path

5.6 Required timing relationships

An `MRA` command does not occupy the data bus. Opposite-view row-path commands may overlap its local occupancy after satisfying `nXACTO`, but ordinary column commands remain subject to shared DDR5 constraints.

5.7 Activation power window

Provisional Each `MRA` counts as one activation event for `nFAW` and the appropriate `nRRD` family. This is a bookkeeping approximation, not a claim that a multi-row activation draws the current of one ordinary activation. A later calibrated model may introduce a weighted activation budget or a dedicated constraint.

6 Prerequisites and State Transitions

6.1 Conventional requests

For a request assigned to view V:

1. If the target row is open in V, issue the terminal column command.
2. If V is closed, issue `ACT_`V, then the column command.
3. If another row is open in V, issue `PRE_`V, `ACT_`V, and then the column command.
4. State in the other view is neither a prerequisite nor a victim.

6.2 Atomic multi-row activation

For `MRA_`V:

1. If V is open, issue `PRE_`V first.
2. The other view need not be closed.
3. On issue, set `busy_until[`V`]` to the current cycle plus `nMRA`.
4. Leave `active_row[`V`]` closed.
5. Do not change the other view's active row or busy deadline.

The controller must preserve an operation's selected view after its first prerequisite issues, following the same commitment principle used for DDRAM-P.

7 Layout-Aware Controller Integration

The initial controller supports host traffic pinned to A and explicit, timing-only MRA requests pinned to B. Layout tags and identity/version are validated, but full operand placement, functional arithmetic, and conversion commands remain future work. The timing ledger remains M1-P1.

7.1 Request contract and ownership

Keep the semantic operation, layout requirement, and current device command separate. A PuD request descriptor must identify its operation, source and destination regions, layout identity/version, element width, bit order, lane mapping, and operand/temporary placement. A layout registry or compiler-owned descriptor may hold this information; each queued primitive need only carry a stable reference plus its decoded target and required view. The exact storage/API for full compound operations remains future work.

The implemented timing-level leaf is `Request::Type::PudMRA`. It carries `required_view=B`, `pud_layout=BitSerial`, nonnegative `layout_id` and `layout_version`, and an already-decoded, concrete B-local address vector. It bypasses the host linear-address mapper so B coordinates cannot be reinterpreted as A coordinates. Missing/incompatible tags, invalid coordinates, and an attempted version change while that layout identity has outstanding PuD leaves are rejected. Identity/version validation does not establish the physical placement or correctness of operand values.

Successful admission returns a shared `pud_completion` token on the submitted request. An optional `pud_dependency` token from an earlier request in the same controller gates a consumer until service completion. The initial API supports one explicit predecessor; a compiler/graph executor will supply more general dependency joins later. Host consumers may also carry this token. Forwarding/coalescing cannot bypass it; a dependency-blocked alias of a buffered write applies admission backpressure until a safe retry. The producer must still describe logical dependencies explicitly; the timing-only controller does not infer shared-cell conflicts from layout IDs.

The layout/operation mapping produces an immutable *required view* for each primitive in the initial integration: A for ordinary host reads/writes, B for bit-serial PuD leaves. This requirement is distinct from the existing `selected_view` and `view_committed` execution fields. An explicit view-local internal command fixes its own view and must agree with any supplied layout requirement. Unknown or contradictory PuD metadata is an error; temporary resource contention is a wait condition. Missing PuD metadata must never silently fall back to the host-access default.

The frontend/PuD compiler supplies the layout contract and operation graph. The address mapper resolves targets in the required view's coordinate system. The controller validates the contract and maps commands; the scheduler ranks eligible requests. The DRAM device remains responsible for prerequisites, state transitions, and timing checks.

7.2 Queue organization and arbitration

Queues are local to one memory channel; there is no PuD queue per bank or per view. The controller uses five schedulable request buffers and two completion-only deques:

Structure	Purpose
`read_buffer`	Admitted host reads.
`write_buffer`	Admitted host writes; existing forwarding and coalescing rules still apply.
`pud_buffer`	Admitted timing-level PuD leaves for all banks in the channel. Its capacity equals `read_buffer_size`, 32 by default.
`priority_buffer`	Controller-issued maintenance and explicit direct commands. MRA is not legal on this path.
`active_buffer`	Host requests that have issued an opening prerequisite and must finish their activation-to-access sequence.
`pending`	Issued reads waiting for their data-return deadline; not schedulable.
`pud_pending`	Issued MRA leaves waiting for internal service completion; not schedulable.

The buffers are insertion-ordered lists, but the schedulable demand and PuD buffers are not strict FIFOs. In each controller cycle, at most one external DRAM command is selected in the following class order:

$$\text{active} > \text{priority} > \text{PuD} > \text{host read/write}. \tag{2}$$

The active class preserves progress for a request whose opening sequence has already begun. The priority class examines only its head. Consequently, a nonempty but timing-blocked priority buffer prevents PuD and host arbitration, although a ready active request may still issue first. Read/write selection retains the existing watermark-controlled write-drain mode.

Within a demand or PuD buffer, FR-FCFS prepares every request, removes ineligible entries from consideration, prefers a timing-ready command, and breaks equal-readiness ties by request arrival time. Eligibility includes an explicit dependency being complete, expiration of any retry delay, managed refresh permission, and protection of in-flight active sequences. Therefore, an older blocked PuD leaf does not prevent an independent younger ready leaf from issuing. If no PuD leaf is issuable, arbitration proceeds to host traffic. `FRFCFS-ViewAware` additionally protects legal view-local row hits from destructive closes; an atomic MRA has no persistent row-hit state, so PuD-leaf ordering normally reduces to readiness and age.

Admission, DRAM readiness, issue, and completion are distinct times. Let t_a be queue admission, t_r the earliest cycle permitted by DRAM timing, t_i actual command issue, and t_c service completion. They obey

$$t_i \geq \max(t_a, t_r), \qquad t_c = t_i + \texttt{nMRA} \tag{3}$$

for the implemented fixed-latency MRA leaf. Scheduler competition may make the first inequality strict. A trace timing arrow records the constraint that determines t_r; it does not assert that the target request was already queued. Visible distance after an arrow's minimum endpoint may therefore be admission slack or arbitration slack rather than an unmodelled device delay.

7.3 PuD queue lifecycle

Admission through `send_pud()` validates the B-local address, required view, bit-serial layout, identity/version, and predecessor token before appending the request to `pud_buffer`. A full buffer applies backpressure. Every scheduler scan then derives the request's current command from the device state:

$$\begin{aligned} \text{B closed:} &\quad \texttt{PudMRA} \rightarrow \texttt{MRA_B}, \\ \text{B open:} &\quad \texttt{PudMRA} \rightarrow \texttt{PRE_B} \rightarrow \texttt{MRA_B}. \end{aligned} \tag{4}$$

An issued `PRE_B` is a prerequisite of the same request. The request stays in `pud_buffer`, retains its decoded coordinates and committed view, and is reconsidered until `MRA_B` becomes ready

after nRP. It does not enter active_buffer: PRE is closing, while MRA is an atomic operation that intentionally leaves no conventional row open.

When the final MRA_B issues at t_i, the request leaves the schedulable buffer and enters pud_pending with departure time t_i+nMRA. Controller view-selection ownership ends at MRA issue, but the device timing ledger prevents same-view use during the remaining service interval. The other view may progress once nXACTO and all shared constraints permit it. At departure, the controller marks the shared completion token complete, releases the outstanding layout reference, updates the served counter, and invokes the callback. A consumer carrying that token is ineligible until this point, not merely until MRA issue. Fixed nMRA makes pud_pending completion-ordered; this deque is never examined by the command scheduler.

PuD counters num_pud_reqs and num_pud_reqs_served count admitted and service-completed leaves, respectively. They do not contribute to read latency or maintenance counts. Direct priority-queue MRA submission is rejected because it would bypass metadata validation, dependency tracking, and delayed service completion. A-view PuD remains deferred.

Cross-view trace example. The development trace intentionally separates phases. Its first PRE_A is admitted only after earlier reads drain, although nRAS made it timing-ready sooner. Likewise, the second MRA_B is admitted after an explicit nRP delay, long after the preceding MRA's nMRA bound has expired. Those gaps are admission slack. The subsequent MRA_B→ACT_A gap is instead tight device timing: ACT_A issues exactly nXACTO cycles after MRA. The example must not be interpreted as evidence that the scheduler held an already-ready PRE or MRA for the entire visible gap.

7.4 Command mapping and scheduling pipeline

The initial mappings are:

- Host read/write: RD_A/WR_A, with local PRE_A/ACT_A prerequisites as needed.
- Bit-serial multi-row primitive: MRA_B, with PRE_B first if its local view is open.
- Explicit B-local read/write: RD_B/WR_B using B-local coordinates and a compatible layout descriptor.
- Future COPY/NOT leaves in the initial PuD integration: use COPY_B/NOT_B when those commands and operand semantics are implemented. A-view PuD leaves remain deferred. The reserved timing parameters alone do not implement these operations.

Extend the existing controller-level prepare_request() path: validate the descriptor, resolve the legal target/view, set final_command, and then ask the device for the next prerequisite. Preparation during scheduler scans must not issue commands, allocate compute rows, mutate layout ownership, or reserve a view. Validate and decode immutable metadata at admission where possible; readiness may change on subsequent scans.

Reuse the IViewSelection boundary for mapping and commitment, with a DDRAM-O layout-constrained policy. Do not simply enable DDRAM-P's TwoView/QueueAware policies for DDRAM-O: those policies assume either view can serve a data request and treat equal row IDs as the same physical row. DDRAM-O uses different coordinate namespaces and layout-pinned targets. No opposite-view row hit, free view, queue preference, or activation NACK may redirect a pinned request to another view.

FR-FCFS continues to rank prepared, dependency-ready requests by readiness and arrival time. A row-hit-preferring scheduler may favor hits only in a request's legal view. It must not choose the layout or translate coordinates. An atomic MRA does not have a persistent open-row hit; its eligibility follows its local prerequisites and timing. A blocked B operation may coexist with independent A progress, subject to shared timing and logical dependencies.

7.5 Commitment, dependencies, and completion

Keep the required view and decoded target stable throughout execution. After the first successfully issued prerequisite, including PRE, commit the selected view and retain it across queue moves and retries. An activation NACK does not commit a previously unstarted request; if PRE already issued, the earlier commitment remains. Any retry uses the same legal view and coordinates. Resource ownership must preserve progress without allowing an incompatible request to invalidate the in-flight prerequisite sequence.

For the first integration, freeze layout/placement while dependent requests are outstanding. Future relocation or conversion must drain or order affected requests and update the layout version explicitly. The operation graph must order conflicting host/PuD accesses and operand producers/consumers; independent view state is not evidence that shared-cell data dependencies are absent. Milestone 1 still omits electrical intersection hazards and functional cell mapping, so timing-only overlap tests cannot establish functional correctness.

Issuing an MRA leaf starts its service interval. A dependent leaf or a parent PuD completion notification must wait for that service to finish, at least nMRA cycles after issue, in addition to graph dependencies. Ordinary read completion retains the existing read-latency path. Do not apply a read data-bus return latency to MRA or report a complete compound operation when its terminal primitive merely issues. COPY/NOT completion will use their own service intervals once implemented.

7.6 Incremental integration and validation

Start with host-only A mapping on DDRAM-O and compare command histories and latency with DDR5. Next inject explicit, layout-pinned B-local MRA requests to validate mapping, PRE/MRA commitment, service completion, and A/B overlap. These are timing-only primitives, not functional arithmetic. Compound bit-serial operations follow once operand placement and the functional model are available. Required first controller tests are:

1. Host read/write traffic uses A and matches DDR5 under equivalent presets, maintenance, row policy, and workload.
2. A B-required PuD request remains on B even when A is idle or has an equal numeric row ID open; a host request likewise stays on A.
3. B-local target coordinates survive prerequisite resolution and queue movement; an open B view causes PRE_B before MRA_B without closing A.
4. Repeated preparation, an issued PRE, and activation retries cannot change the required view, descriptor version, or decoded target.
5. Missing, incompatible, or contradictory PuD layout metadata is rejected; occupied resources wait without triggering an implicit conversion. A-view PuD requests are rejected as outside the initial controller scope, while ordinary A-view host accesses remain supported.
6. Dependent operations and completion wait for MRA's full service interval; independent A work can overlap when shared constraints allow.
7. Conventional DDR5 and DDRAM-P controller regressions remain unchanged.

8 Configuration Interface

The Python DRAM specification must accept explicit overrides. A representative configuration is:

```
ramulator.dram.DDRAM_O(
    org_preset="DDR5_16Gb_x8",
    timing_preset="DDR5_3200AN",
    rank=1,
    nXACTO=8,
    nMRA=76,
    nCOPY=83,
    nNOT=83,
)
```

The example explicitly supplies the M1-P1 split-profile values; omitting those primitive overrides now derives the same defaults. Derived defaults must be resolved after the inherited DDR5 preset and base overrides so that changing `nRAS` or `nRP` changes the placeholder consistently. Decoder delay must remain a configurable physical-time assumption, initially 4 ns. The implemented constructor accepts `tDEC_ns=4.0` and `copy_timing_profile="pumulator-split"`; the alternative is `"pumulator-two-activate"`. Both use ceiling rounding. Configuration metadata in `ddram_timing_model` records the ledger revision, copy profile, decoder delay, presets, explicit overrides, and intersection flag. Every experiment result must record:

- the resolved values of `nXACTO`, `nMRA`, `nCOPY`, and `nNOT`;
- the DDR5 organization and timing presets;
- the copy profile (split or two-activate), decoder delay in ns, clock period, rounding rule (ceiling), and any explicit overrides;
- a timing-model revision string, initially `M1-P1`; and
- whether intersection modeling is enabled, initially `false`.

Invalid negative or zero DDRAM-O-specific timing values must be rejected after default resolution. The decoder-delay input and clock period must be finite and positive.

Implementation alignment. The simulator implements M1-P1 defaults, both copy profiles, and PuD-default derivation after explicit base/primitive overrides, locally to DDRAM-O. An unoverridden NOT follows the final COPY value; MRA overrides do not change COPY. Conventional DDR5 parameter resolution is unchanged. COPY/NOT remain reserved costs rather than issued commands. Runs must record actual resolved values and distinguish timing-only leaves from functional compound operations.

9 Trace and Statistics Requirements

The existing trace recorder must represent every DDRAM-O command without changing conventional trace semantics. Each event should expose:

- command name and issue cycle;
- selected view;
- bank/subarray and representative row address;
- whether it is a conventional or PuD command;
- the timing constraint that determined readiness; and
- the resolved timing value used by that constraint.

The initial statistics should include counts of view-A and view-B activations, multi-row activations by view, cycles each view is busy, cross-view overlap cycles, and stalls attributed to `nXACTO` or `nMRA`.

10 Verification Plan

Milestone 1 is complete only when the following tests pass.

Every timing test should independently replay the issued history through the device prerequisite and timing checks, as the DDRAM-P suite already does.

11 Calibration and Evolution

The command interface is intended to remain stable while the physical model becomes more detailed.

11.1 SPICE calibration

The first calibration step replaces `nXACTO` using the minimum safe separation between opposite-view sensing events under the selected voltage, temperature, process corner, and row multiplicity. The report should provide the time in nanoseconds and the conversion rule

$$\texttt{nXACTO} = \left\lceil \frac{t_{\mathrm{XACTO}}}{t_{\mathrm{CK}}} \right\rceil . \tag{5}$$

Multi-row activation simulations should subsequently calibrate local-view occupancy, restoration time, current limits, and whether MRA should carry more than one activation unit in the power window.

Each calibration update must increment the timing-ledger revision, retain the superseded defaults in version history, and record the circuit configuration, process/voltage/temperature corner, row multiplicity, time in nanoseconds, clock period, rounding rule, and resulting cycle count. This makes results produced with provisional and calibrated models distinguishable.

11.2 Intersection-aware extension

A later revision may add intersection identity, masking, and simultaneous sensing restrictions. It should extend readiness checks and functional data mapping, not replace view-specific command names or provisional timing parameters. Experiments must report which revision they use.

11.3 Functional PuD extension

Milestone 2 will attach explicit operand and destination rows to compound PuD requests. A functional shadow array will map the orthogonal coordinate systems onto shared cells and validate majority, addition, and multiplication. The atomic `MRA_B` command defined here is the initial multi-row timing leaf of those operation graphs; A-view PuD remains a deferred extension.

11.4 Intermediate-result cost accounting

Pumulator's `Model::build_simdram()` provides a reproducible baseline for future graph costs. Its source is:
`simulator/ramulator2/src/frontend/impl/processor/simple03/pum_model.cpp`

Its default MAJ3 sequence copies three operands into compute rows, executes one AP, and copies one result out. Mapping AP to MRA gives the serialized service cost

$$n_{\mathrm{MAJ3}} = 3\texttt{nCOPY} + \texttt{nMRA} + \texttt{nCOPY}. \tag{6}$$

At DDR5_3200AN under the split profile this is 408 cycles (255 ns), before extra inversion, layout conversion, movement between views, queueing, maintenance, and shared-resource contention. This is a primitive-sequence estimate, not a simulated end-to-end latency or the cost of one MRA alone. The final precharge is already included in each primitive's occupancy; an initially open local view still needs its ordinary prerequisite precharge.

Milestone 2 must retain dependency edges and operand lifetimes, track which values are overwritten by destructive multi-row sensing, and account for copies that preserve live intermediate values. Copy elision requires proven residency and liveness; a value in A is not automatically available in B at zero cost. Fused or free inversion requires a supported physical path and placement. Pumulator's distance-based spill heuristic can inform experiments but is not a substitute for an allocator with explicit

Table 7: Acceptance tests.

ID	Scenario	Required observation
S1	Open one row through each view.	Both rows remain open independently.
S2	Precharge one view.	Only that view closes.
S3	Open equal view-local line indices in A and B.	Both activations are accepted; equality of namespace-local labels creates no Milestone 1 intersection rule.
T1	Issue opposite-view activations.	Start times differ by at least `nXACTO`.
T2	Repeat T1 with an override.	The observed separation follows the configured value exactly when no stricter constraint applies.
T3	Issue `MRA_A`, then an A operation.	The A operation waits at least `nMRA`.
T4	Issue `MRA_A`, then a B operation.	B may start after cross-view constraints without waiting for the full `nMRA`.
T5	Exercise the four-activation window.	MRA contributes one provisional activation event.
P1	Issue MRA while its local view is open.	A local precharge is issued first; the other view remains open.
P2	Issue rank or same-bank maintenance.	Both views close before inherited maintenance executes.
C1	Run conventional DDR5 tests.	Command translation, timing, and state behavior are unchanged.
C2	Record a mixed A/B/MRA sequence.	Trace contains view identity and the expected timing dependencies.
D1	Resolve the split profile at DDR5_3200AN.	MRA is 76 cycles, COPY and unoverridden NOT are 83 cycles; the decoder delay is rounded up from 4 ns to 7 cycles.
D2	Change speed bin, decoder delay, or base timings.	Defaults follow the physical-time formulas after base overrides, with ceiling conversion; two-activate COPY uses 2`nRAS` $+$ `nRP`.
D3	Override MRA, COPY, and NOT independently.	MRA alone does not alter COPY; an unoverridden NOT follows resolved COPY; explicit NOT wins. Invalid values are rejected.

row capacity. Schedule the resulting primitives through actual view, command-bus, and activation constraints; do not divide serialized costs by assumed bank or subarray parallelism factors to claim DDRAM-O latency.

12 Implementation Boundary

The recommended separation of responsibilities is:

- **DRAM specification:** commands, state transitions, prerequisites, timing constraints, and parameter resolution;
- **controller:** request preparation, selected-view commitment, timing-aware issue, and retirement;
- **PuD compiler (later):** operation decomposition, dependencies, operand placement, and temporary rows;
- **scheduler (later):** selection among ready primitive commands;
- **functional model (later):** data-value correctness; and
- **energy model (later):** calibrated per-primitive energy.

This boundary preserves conventional DDR5 behavior and prevents placeholder electrical assumptions from becoming embedded in scheduling policy.